Sakthy learns Engineering

December 4, 2025

Contents

1 Python Fundamentals

1.1 Modules and Packages

1.1.1 What's in `__init__.py`?

`__init__.py` is used to mark a directory as a Python package. There seems to be a lot of confusion about what should go inside this file. Some say it needs to be empty and some say we can put some there. But for me (*after lots of reading*), it felt like `__init__.py` should not be empty but should not contain too much code either.

It can contain the following:

- **Package Initialization:** Basically things you want to run when the package is imported. This can include logging, configuration settings, etc.
- **Act as public API:** You can define what is accessible when the package is imported using `__all__` list. So someone who is using this package can know what is the public interface rather than digging into the package internals.
- **Submodule Imports:** This is important for backward compatibility. If you have submodules in your package and you want to make them accessible directly from the package, you can import them in `__init__.py`. This way, users can access submodules without needing to know the internal structure of the package.

But it should **NOT** contain:

- **Heavy Business Logic:** I don't think we should put heavy business logic or complex functions in `__init__.py`. This should be purely an interface file rather than a place to put all the code.
- **Large Imports:** Avoid importing large libraries or modules in `__init__.py` as it can slow down the package import time. Only import what is necessary for the package initialization and what you want to expose as part of the public API.
- **Circular Imports:** Be cautious about importing modules that might lead to circular dependencies. This can cause import errors and make the code harder to maintain.

1.1.2 How does package gets imported?

When you import a package in Python, the interpreter follows a specific sequence of steps to locate and load the package. Here's a simplified explanation of how it works:

1. **Search in sys.modules:** First, Python checks if the package is already loaded in memory by looking in the `sys.modules` dictionary. If it finds the package there, it uses the existing module object.
2. **Locate the Package:** If the package is not found in `sys.modules`, Python searches for the package in the directories listed in `sys.path`. This list includes the current directory, standard library directories, and any directories specified in the `PYTHONPATH` environment variable.
3. **Load the Package:** Once Python locates the package directory, it looks for the `__init__.py` file inside that directory. This file is executed to initialize the package.

4. **Create Module Object:** Python creates a new module object for the package and adds it to `sys.modules`.

5. **Execute `__init__.py`:** The code inside `__init__.py` is executed. This can include setting up package-level variables, importing submodules, and defining the package's public API.

6. **Make the Package Available:** After executing `__init__.py`, the package is now available for use in the importing module. You can access its submodules and attributes as defined in `__init__.py`.

So many steps just to import a package! *So remember, don't ignore the linter error of unused imports lol. The app is dying from all these unnecessary imports slowing down the import time.*

1.1.3 Absolute vs Relative imports

Absolute imports is just specifying the full path of the module you want to import starting from the project's root directory. For example:

```
from my_package.submodule import my_function
```

Listing 1: Absolute Import Example

Relative imports, on the other hand, use dot notation to specify the location of the module relative to the current module. For example:

```
from .submodule import my_function  # Single dot means current package
from ..another_module import another_function  # Double dot means parent
    package
```

Listing 2: Relative Import Example

I see more value in absolute module because it remove cognitive load. It's all just there. You know exactly where the module is located in the project structure. There is no need for any mind calculations. But it's not bells and whistles. Relative imports to certain extent decouple the module from the project structure. So if you are moving modules around a lot, relative imports can be useful. But I think absolute imports are the way to go for most cases.

2 Database

2.1 Understanding SQL query cost

Why do need optimized SQL queries? Why can't we just write any query and let the database handle it? So it seems that there seems to be a cost associated with executing SQL queries. This cost can be measured in terms of time taken to execute the query, resources consumed (like CPU and memory), and the impact on other operations happening in the database.

From what I have experienced so far, the cost of SQL queries is made up of (within things that you can control):

- **Data Volume:** Duh. The more data you have to process, the longer it takes.
- **Query Complexity:** If an SQL query is doing way too much work in a single statement, it can be slow.
- **Index Usage:** Lack of indexes they say but its not that simple. Double edged sword. Indexes speed up read operations but can slow down write operations.
- **Locking and Concurrency:** When we are running multiple queries at the same time, they can interfere with each other. Locks and delays can add to the cost.
- **Network Latency:** If the database is on a different server, the time taken to send and receive data over the network can add to the overall cost. Usually bottleneck when the connection pool is exhausted sometimes.

2.1.1 Case 1: EXPLAIN ANALYZE statement

I have seen the **EXPLAIN ANALYZE** statement in SQL but do I understand it properly? *Heck no.* So I'm stuck in a solution where I have to optimize queries but I don't know how to do it properly. Well I have written a monstrousity of a query to export data from MySQL to CSV files. It works and accomplishes the task.

```
WITH RECURSIVE city_items AS (
    SELECT
        lr.id AS location_restriction_id,
        lr.description,
        'origin_multiple_cities' AS field_name,
        lr.origin_multiple_cities AS original_cities_csv,
        TRIM(SUBSTRING_INDEX(SUBSTRING_INDEX(lr.origin_multiple_cities, ',
            ', numbers.n), ',', -1)) AS item_value,
        numbers.n
    FROM
        rate_card_locationrestriction lr
    CROSS JOIN (
        SELECT 1 AS n UNION ALL SELECT 2 UNION ALL SELECT 3 UNION ALL
            SELECT 4 UNION ALL SELECT 5
        UNION ALL SELECT 6 UNION ALL SELECT 7 UNION ALL SELECT 8 UNION ALL
             SELECT 9 UNION ALL SELECT 10
        UNION ALL SELECT 11 UNION ALL SELECT 12 UNION ALL SELECT 13 UNION
            ALL SELECT 14 UNION ALL SELECT 15
        UNION ALL SELECT 16 UNION ALL SELECT 17 UNION ALL SELECT 18 UNION
            ALL SELECT 19 UNION ALL SELECT 20
```

```
    ) numbers
    WHERE
        lr.origin_multiple_cities IS NOT NULL
        AND lr.origin_multiple_cities != ''
        AND numbers.n <= 1 + (LENGTH(lr.origin_multiple_cities) - LENGTH(
            REPLACE(lr.origin_multiple_cities, ',', '')))
)
SELECT
    si.location_restriction_id,
    si.description,
    c.id AS company_id,
    c.company_name,
    rc.id AS rate_card_id,
    rc.name AS rate_card_name,
    si.field_name,
    si.item_value,
    si.original_cities_csv,
    CASE
        WHEN COUNT(DISTINCT s1.id) > 0 OR COUNT(DISTINCT s2.id) > 0 THEN '
            Yes'
        ELSE 'No'
    END AS found_in_lookup,
    COALESCE(GROUP_CONCAT(DISTINCT s1.id ORDER BY s1.id SEPARATOR ','), ''
        ) AS matched_by_name_ids,
    COALESCE(GROUP_CONCAT(DISTINCT s2.id ORDER BY s2.id SEPARATOR ','), ''
        ) AS matched_by_code_ids
FROM
    city_items si
LEFT JOIN
    rate_card_ratecard_location_restrictions rclr ON rclr.
        locationrestriction_id = si.location_restriction_id
LEFT JOIN
    rate_card_ratecard rc ON rc.id = rclr.ratecard_id
LEFT JOIN
    company_company c ON c.id = rc.shipper_company_id
LEFT JOIN
    common_cityv2 s1 ON LOWER(s1.name) = LOWER(si.item_value)
LEFT JOIN
    common_cityv2 s2 ON LOWER(s2.name_wo_diacritics) = LOWER(si.item_value
        )
WHERE
    TRIM(si.item_value) != '' and rc.shipper_company_id = '572'
GROUP BY
    si.location_restriction_id,
    si.description,
    c.id,
    c.company_name,
    rc.id,
    rc.name,
    si.field_name,
    si.item_value
ORDER BY
    c.company_name,
    rc.name,
```

```
64      si.location_restriction_id;
```

Sorry for abusing your eyes with this monstrous query. What this query does is that it extracts individual city names from a comma-separated list of cities stored in the `origin_multiple_cities` field of the `rate_card_locationrestriction` table. It then checks if these city names exist in the `common_cityv2` table either by name or by name without diacritics. Finally, it aggregates the results to show which cities were found and their corresponding IDs. The query seems bad but how bad it is?

Analyzing the EXPLAIN output So I know what it does, but what's the damage? So hence, let's run `EXPLAIN ANALYZE` on this query. The output is a below:

```
-> Sort: c.company_name, rc.‘name‘, si.location_restriction_id
   (actual time=1593..1593 rows=4 loops=1)
    -> Stream results  (actual time=1593..1593 rows=4 loops=1)
        -> Group aggregate: group_concat(distinct common_cityv2.id ...)
           (actual time=1593..1593 rows=4 loops=1)
            -> Sort: si.location_restriction_id, si.‘description‘, ...
               (actual time=1593..1593 rows=43 loops=1)
                -> Stream results  (cost=62.3e+12 rows=623e+12)
                   (actual time=1406..1593 rows=43 loops=1)
                    -> Left hash join (lower(s2.name_wo_diacritics)=...)
                       (cost=62.3e+12 rows=623e+12)
                       (actual time=1406..1593 rows=43 loops=1)
                        -> Left hash join (lower(s1.‘name‘)=...)
                           (cost=70.5e+6 rows=705e+6)
                           (actual time=616..813 rows=13 loops=1)
                            -> Filter: (trim(...) <> ’’ and ...)
                               (cost=577 rows=797)
                               (actual time=1.59..1.66 rows=4 loops=1)
                                -> Inner hash join (no condition)
                                   (cost=577 rows=797)
                                   (actual time=1.58..1.6 rows=80 loops=1)
                                    -> Table scan on numbers
                                       (actual time=0.0314..0.0339 rows=20)
                                    -> Hash
                                        -> Nested loop inner join
                                           (actual time=0.185..1.51 rows=4)
                                            -> Filter: origin_multiple_cities
                                               IS NOT NULL
                                               (actual time=0.00221..0.00221
                                               rows=0.0132 loops=302)
                            -> Hash
                                -> Table scan on s1
                                   (cost=122 rows=884073)
                                   (actual time=0.118..270 rows=890564)
                        -> Hash
                            -> Table scan on s2
```

```
(cost=10.9 rows=884073)
(actual time=0.0835..275 rows=890564)
```

So the output is tree-structure like and needs to be read from innermost to outermost. Some key metrics to remember are:

- **actual time:** Time in milliseconds
  - Format: `actual time=start..end`
- **rows:** How many rows were processed
- **cost:** Estimated abstract unit for estimating query expensive
- **loops:** How many times this operation ran

Understanding "cost" Cost is a weighted sum of various operations. For example, MySQL calculates it like this.

$$\begin{aligned}\texttt{cost} = &(\texttt{rows\_examined} \times \texttt{row\_eval\_cost}) \\ &+ (\texttt{pages\_read} \times \texttt{io\_block\_read\_cost})\end{aligned}$$

where each cost as a weight associated to it. So the "cost" is relative. It has no real value. But it abstractly show you how expensive an operation is.

So what does this mean for my query? The flow seems to be like this:

1. Scans 890K cities twice (540ms)
2. Builds hash tables for lookups (616-1406ms)
3. Processes only 4 location restrictions that match the filter
4. Returns 43 intermediate rows, grouped down to 4 final rows
5. Total execution time: 1.6 seconds

So these kinda reveal some serious issues:

- No index usage because of `LOWER()` functions on both sides of joins
- Full table scans on a 890K row table (twice!)
- Plan shows 62.3e+12 cost (12 trillion!)
- The actual data needed is tiny (4 restrictions, 4 cities) but MySQL doesn't know this upfront and must scan everything

So `EXPLAIN ANALYZE` statement can kinda you give you understanding how good or bad query is (*although I do not yet understand it fully, just minimal understanding alone can you give you so much context*). For my use-case, this is an ad-hoc script that I'm gonna run once to generate a CSV report. So I don't need to fix this. Yay! *PS: Please do fix this if its a query that you will run more than once.*